Read-Through and Write-Through

let the cache load and store data itself

Watch the 4-min video

In one line: in read-through the cache itself loads missing data from the database through a configured loader, so the app only talks to the cache; in write-through every write goes through the cache, which writes the database synchronously before confirming, keeping them in sync at the cost of slower writes and caching data nobody reads.

1 The same code in every service

With cache-aside (lesson 16), every service writes the same code: check the cache, query the database, fill the cache. What if the cache did that work itself? You'd talk only to the cache, and it would keep itself filled and in sync. That's **read-through** and **write-through** caching.

2 Read-through

The application reads **only from the cache**. On a miss, the cache itself loads the data from the database, using a **loader** function you configure, stores it and returns it. The miss logic lives in one place, the cache provider, instead of being repeated in every service.

The flow on a miss is the same as cache-aside; what changes is **who runs it**. In cache-aside the application owns the logic and can use any cache. In read-through the cache owns it, so app code is simpler, but you need a cache that supports loaders: Caffeine or Guava `LoadingCache` in Java, Ehcache, or a managed product such as Amazon DAX. A nice bonus: a good loader coalesces concurrent misses for the same key into one database call.

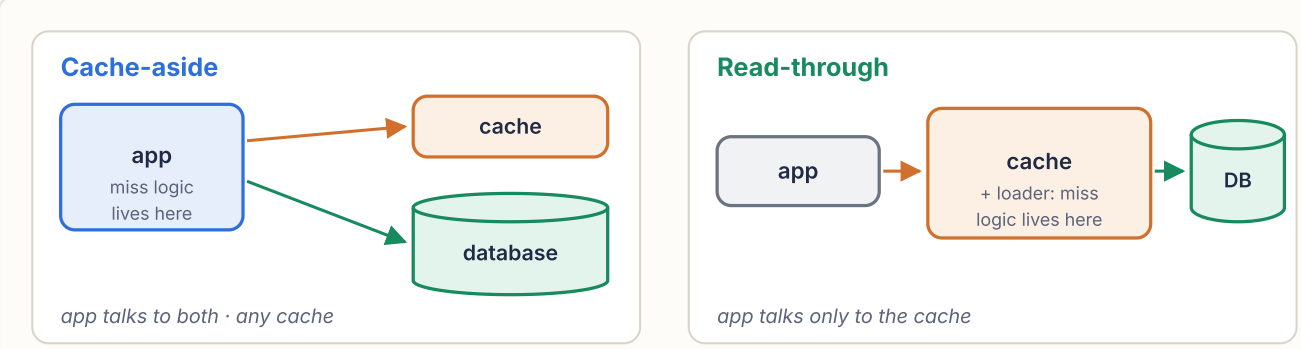


Figure 1. Same miss flow, different owner: in cache-aside the app runs it, in read-through the cache's loader does.

3 Write-through

Every write goes **to the cache**, and the cache **synchronously** writes it to the database before confirming. The write isn't done until both are updated. So the cache is always in sync with the database, and a read right after a write sees the new value, a cache hit.

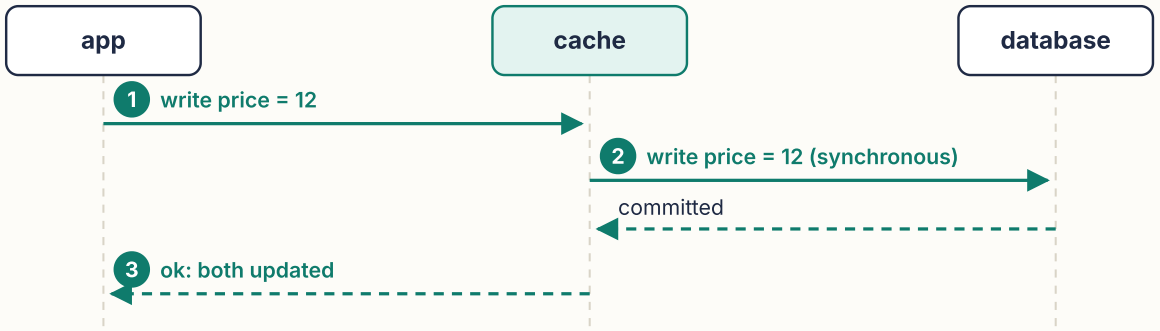


Figure 2. Write-through: the cache forwards the write to the database and confirms only after both are updated.

THE COST OF WRITE-THROUGH

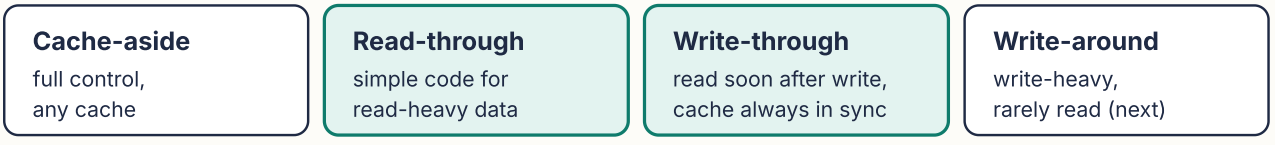
- **Higher write latency**: every write pays for two writes, cache and database, in sequence.
- **Cache pollution**: data that's written but never read still fills the cache. Pair it with TTLs (and eviction) so unused entries leave.
- **Partial failure**: if the database write fails after the cache write, the provider must roll back or invalidate the cache entry, or the two disagree.

Write-through is usually paired with read-through: writes keep the cache fresh, and reads fill in whatever is missing.

WORKED EXAMPLE: DAX IN FRONT OF DYNAMODB

DAX, the DynamoDB Accelerator, is an in-memory cache that speaks the DynamoDB API. The application swaps the DynamoDB client for the DAX client and changes nothing else. **Reads are read-through**: a hit returns in microseconds, a miss is fetched from DynamoDB and cached. **Writes are write-through**: DAX writes to DynamoDB, then updates its item cache. There's no cache code in the application at all. (One subtlety: strongly consistent reads bypass DAX and go straight to DynamoDB.)

4 Choose by workload



most teams default to cache-aside; providers like DAX give read-through + write-through for free

Figure 3. Four patterns and what each one suits.

- **Cache-aside**: maximum control, works with any cache. The common default.
- **Read-through**: simpler code for read-heavy data, when your cache supports loaders.
- **Write-through**: data that's read soon after it's written and must be consistent in the cache, such as a user's profile after they edit it.
- For write-heavy data that's rarely read, write-through wastes effort; **write-around** is better (next lesson).

COMMON MISTAKES

- Calling write-through "consistent" while other services write to the database directly and bypass the cache.
- Write-through with no TTL, so the cache fills with write-only data.
- Ignoring the failure case where the cache write succeeds and the database write fails.

5 Key terms

Read-through: the cache loads misses itself

Write-through: writes go through the cache to the DB, synchronously

Cache pollution: entries that are never read taking space

Loader: the function the cache calls on a miss

DAX: DynamoDB Accelerator, a read/write-through cache

Request coalescing: one load for many concurrent misses

6 Interview questions

Q1. Compare cache-aside with read-through. BASIC

In cache-aside the application checks the cache, loads from the database on a miss and fills the cache itself; in read-through the cache provider does that loading, so application code only talks to the cache.

Q2. What is the cost of write-through? BASIC

Higher write latency, because each write synchronously updates both cache and database, plus caching data that may never be read.

Q3. What's the benefit of write-through? BASIC

The cache always holds the latest value for written keys, so reads right after writes are hits and never stale, as long as all writes go through the cache.

Q4. Users edit their profile and immediately view it. Which pattern? SCENARIO

Write-through (with read-through for misses): the edit updates the cache and database together, so the next view is a fresh hit. With cache-aside, delete-on-write followed by a miss also works but costs a database read.

Q5. You use DynamoDB and want microsecond reads with no cache code. What do you use? SCENARIO

DAX: swap in the DAX client for read-through and write-through caching, and keep strongly consistent reads (which bypass DAX) for the few cases that need them.

Q6. Can write-through still serve stale data? TRICKY

Yes, if anything writes to the database without going through the cache (a batch job, another service, a manual fix), or if the cache entry was evicted and refilled from a lagging replica. TTLs limit the damage.

Next: SD-F-004 · Write-Behind and Write-Around backendroadmap.com